Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

ЗВІТ

з лабораторної роботи № 3

з дисципліни «Теорія прийняття рішень»

Тема: «Агрегування та факторизація»

Варіант № 1

Виконав:
студент групи ФЕІ-34с
Біжик Юрій Володимирович

Перевірила:
доцент Дуфанець Марта
Василівна

Львів — 2026

1 Мета роботи

Засвоїти агрегування та факторизацію бінарних відношень. Реалізувати булеву композицію, транзитивне замикання, відношення досяжності й взаємної досяжності, побудувати класи еквівалентності та граф факторизованого відношення.

2 Завдання, варіант та вихідні дані

Номер студента у журналі N = 1, тому кількість циклічних зсувів елементів вліво становить N − 1 = 0. Використано матрицю з пункту 2 порядку виконання наданої методички `materials/Lab3.pdf`: окремої матриці від викладача не надано. Для варіанта 1 зсув відсутній, тому результат не залежить від того, чи трактувати зсув як зсув рядків або суцільної послідовності елементів. Програма підтримує обидва режими; типовий `flat` зсуває послідовність, отриману читанням матриці по рядках, режим `rows` зсуває кожен рядок окремо.

Носій відношення — $A = \{x1, x2, x3, x4\}$. Елемент $p_{ij} = 1$ означає, що $x_i \, P \, x_j$, тобто існує орієнтована дуга з x_i до x_j. Петлі враховуються нарівні з іншими дугами.

	x1	x2	x3	x4
x1	1	0	0	1
x2	0	0	1	0
x3	1	1	0	0
x4	0	0	1	1

Таблиця 1 — Матриця початкового бінарного відношення P

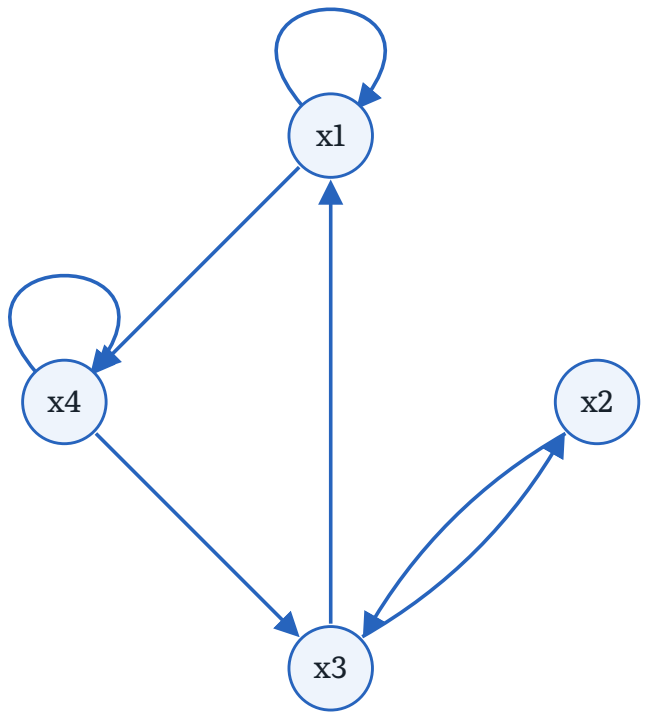


Рисунок 1 — Граф початкового відношення P зі збереженням петель

У графі є дуги x1→x1, x1→x4, x2→x3, x3→x1, x3→x2, x4→x3, x4→x4. Взаємні дуги між x2 і x3 подано окремо. Матриця не є рефлексивною, оскільки $p_{22} = p_{33} = 0$; не є симетричною, оскільки x1 P x4, але x4 P x1 відсутня. Вона також не транзитивна: x2 P x3 і x3 P x1, але x2 P x1 відсутня.

3 Теоретичні відомості

Булева композиція двох відношень на спільному носії задається правилом:

$$(P \circ Q)_{ij} = 1 \Leftrightarrow \exists k \in \{1, \ldots, n\} : p_{ik} = 1 \wedge q_{kj} = 1. \tag{1}$$

Використовується логічне «існує», тому звичайне арифметичне множення матриць без перетворення ненульових результатів на одиниці не відповідає бінарній композиції. Матриця P^k описує наявність шляху з k дугами. Транзитивне замикання містить усі пари, пов'язані шляхами додатної довжини:

$$P^{+} = \cup_{k \geq 1} P^{k}. \tag{2}$$

Послідовність булевих степенів скінченної матриці рано чи пізно повторюється. Якщо $P^t = P^s$, то подальші множення на P повторюють уже пройдену частину послідовності. Тому для повного об'єднання досить степенів до першого повторення. Важливо виявляти повторення **будь-якого** попереднього степеня, а не лише рівність двох сусідніх: для відношення з періодичними циклами сусідні матриці можуть не збігатися.

Досяжність допускає шлях нульової довжини:

$$R = E \cup P^{+}, \quad D = R \cap R^{T}. \tag{3}$$

E — одиничне відношення, а R^T — обернене відношення. D задає взаємну досяжність та є еквівалентністю. Її класи відповідають компонентам сильної зв'язності орієнтованого графа. Факторизація переносить P на множину цих класів:

$$A_i \left(\frac{P}{D}\right) A_j \Leftrightarrow \exists x \in A_i, \exists y \in A_j : xPy. \tag{4}$$

Отже, фактор-матриця обчислюється за існуванням хоча б однієї первісної дуги між відповідними класами, а не за кількістю дуг і не за всіма парами одночасно.

4 Послідовність обчислення

4.1 Булеві степені та транзитивне замикання

Перший степінь наведено у вихідних даних. Програма послідовно обчислила наступні степені:

	x1	x2	x3	x4
x1	1	0	1	1
x2	1	1	0	0
x3	1	0	1	1
x4	1	1	1	1

Таблиця 2 — Булевий степінь P^2

	x1	x2	x3	x4
x1	1	1	1	1
x2	1	0	1	1
x3	1	1	1	1
x4	1	1	1	1

Таблиця 3 — Булевий степінь P^3

	x1	x2	x3	x4
x1	1	1	1	1
x2	1	1	1	1
x3	1	1	1	1
x4	1	1	1	1

Таблиця 4 — Булевий степінь P^4

	x1	x2	x3	x4
x1	1	1	1	1
x2	1	1	1	1
x3	1	1	1	1
x4	1	1	1	1

Таблиця 5 — Булевий степінь P^5

Перше повторення зафіксовано на степені 5: він збігається зі степенем 4. Період повторення дорівнює 1. У цьому наборі $P^4 = P^5$ — матриця з усіма одиницями. Об'єднання $P \cup P^2 \cup P^3$ уже також містить усі пари; подальші степені не додають нових елементів. Критерій зупинки відповідає вимозі методички шукати найменший порядок композиції, за якого повторюється матриця.

	x1	x2	x3	x4
x1	1	1	1	1
x2	1	1	1	1
x3	1	1	1	1
x4	1	1	1	1

Таблиця 6 — Транзитивне замикання P^+ як об'єднання обчислених степенів

4.2 Досяжність та взаємна досяжність

	x1	x2	x3	x4
x1	1	1	1	1
x2	1	1	1	1
x3	1	1	1	1
x4	1	1	1	1

Таблиця 7 — Матриця досяжності $R = E \cup P^{+}$

	x1	x2	x3	x4
x1	1	1	1	1
x2	1	1	1	1
x3	1	1	1	1
x4	1	1	1	1

Таблиця 8 — Матриця взаємної досяжності $D = R \cap R^{T}$

У конкретному варіанті всі три матриці P^{+}, R і D складаються з одиниць. Досяжність є повною: із кожної вершини можна потрапити до будь-якої іншої. Зокрема, замкнений маршрут x1→x4→x3→x2→x3→x1 проходить через всі чотири вершини. Це пояснює належність усіх вершин до одного класу взаємної досяжності.

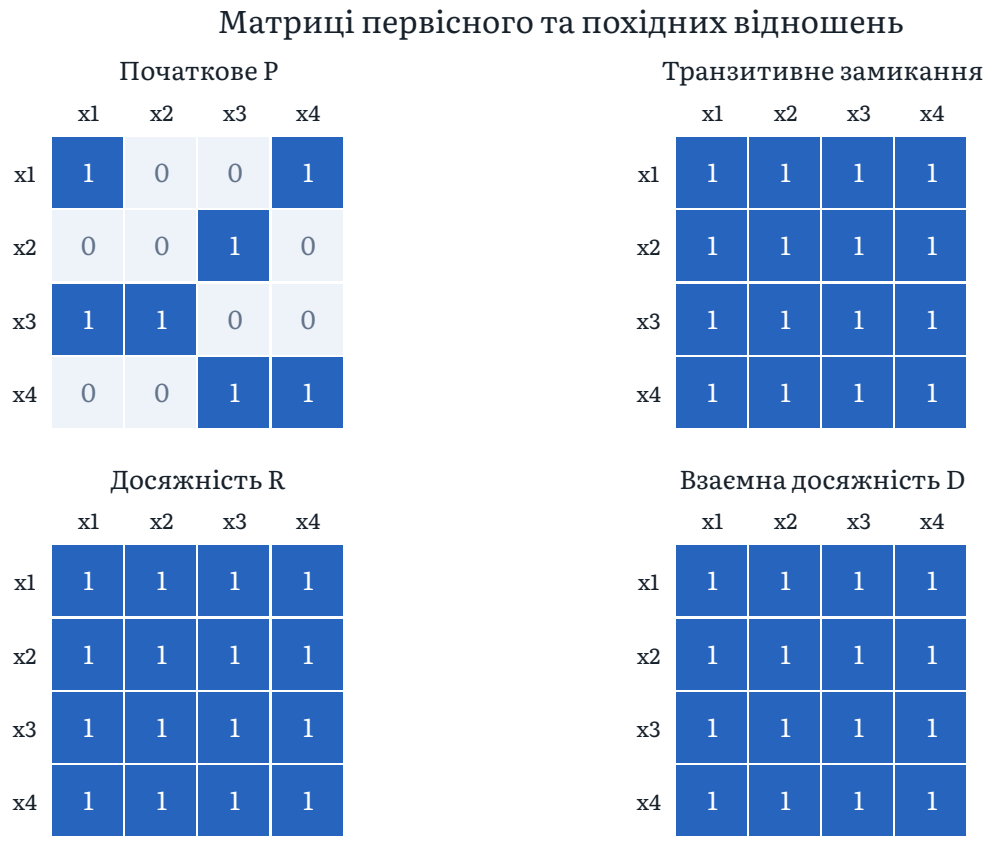


Рисунок 2 — Порівняння первісної матриці та матриць замикання, досяжності й взаємної досяжності

Синя клітинка означає одиницю, світла — нуль; числа в клітинках зберігають точний зміст. Рисунок показує, як відсутні первісні дуги доповнюються парами, для яких існує складений шлях.

4.3 Класи еквівалентності та фактор-відношення

Клас	Вершини початкового графа
A1	x1, x2, x3, x4

Таблиця 9 — Класи взаємної досяжності

Одержано 1 клас: A1 = {x1, x2, x3, x4}. Вся множина-носій стала одним агрегованим елементом. Фактор-відношення містить пару (A1, A1), оскільки в початковому графі є дуги всередині цього класу.

	A1
A1	1

Таблиця 10 — Матриця фактор-відношення P/D за екзистенційним правилом

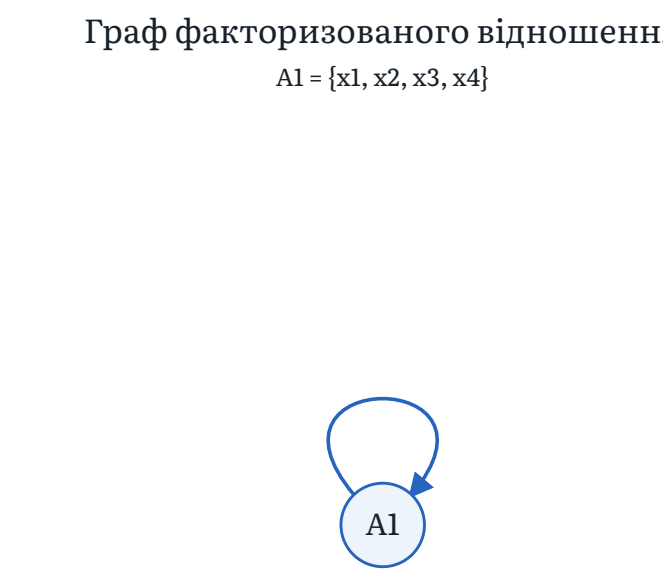


Рисунок 3 — Граф фактор-відношення: один агрегований клас із петлею

Петля не означає, що між різними компонентами лишився цикл. У визначенні фактор-відношення з методички не заборонено i = j, тому внутрішні дуги дають діагональну одиницю. Для звичайної ациклічної конденсації сильнозв'язних компонент такі внутрішньокласові петлі вилучаються. Це розмежовує буквальну фактор-матрицю та DAG-конденсацію, яку методичка описує як граф без контурів.

	A1
A1	0

Таблиця 11 — Матриця конденсації після вилучення внутрішньокласових петель

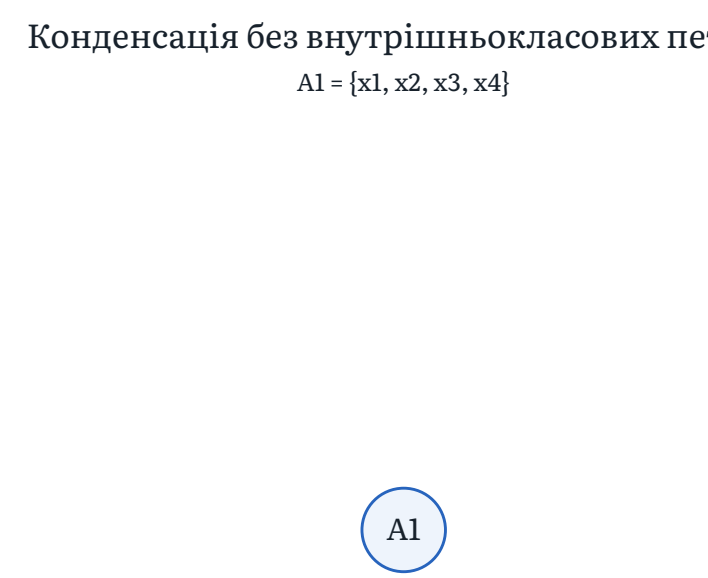


Рисунок 4 — Ациклічна конденсація: єдина вершина без дуг

У цьому варіанті P/D = (1), а конденсація дорівнює (0). Їх не можна підміняти одна одною без пояснення. Відношення P/D є рефлексивним, транзитивним та антисиметричним на одноелементному носії; петля відповідає рефлексивності.

5 Програмна реалізація

`src/main.py` використовує лише стандартну бібліотеку Python. `composition` реалізує булеву композицію, `solve` формує степені, їх об'єднання, R та D, класи й фактор-матрицю. Поточний степінь представлений незмінним ключем у словнику вже отриманих матриць, тому перше повторення визначається однозначно. За замовчуванням установлена явна межа 10 000 степенів: якщо повторення за цей час не знайдено, програма повідомляє помилку та пропонує збільшити `--max-powers`, а не видає неповне замикання за результат.

Витрати однієї булевої композиції — $O(n^3)$, пошук пар і факторизація — $O(n^2)$. Якщо до першого повторення обчислено t степе-

нів, час становить $O(tn^3)$, а пам'ять для історії степенів — $O(tn^2)$. `src/charts.py` генерує SVG без сторонніх бібліотек. `data/results.json` містить повний перебіг обчислення, CSV — окремі матриці та класи; `index.html` відображає таблиці та графи.

Команда виконання з кореня репозиторію:

```
uv run python labs/lab-03/src/main.py --student-number 1
```

У звіті замість знімків екрана наведено точні числові таблиці й векторні рисунки відповідно до прямої вказівки користувача. Код нижче є лістингом реалізації, а не зображенням.

6 Висновки

Для варіанта N=1 реалізовано повну послідовність факторизації бінарного відношення: булеві степені, транзитивне замикання, досяжність, взаємну досяжність, класи еквівалентності та фактор-матрицю. Першим повторюється п'ятий степінь, який дорівнює четвертому. Всі чотири вершини належать одній сильнозв'язній компоненті, тому факторизація зменшує носій із чотирьох елементів до одного.

Буквальне екзистенційне фактор-відношення має петлю та матрицю (1). Після вилучення внутрішньокласової петлі одержано ациклічну конденсацію з однією вершиною й матрицею (0). Такий результат показує, що агрегування приховує внутрішні цикли всередині класу, зберігаючи зв'язки між класами.

7 Використані матеріали

1. Методичні матеріали до лабораторної роботи №3 «Агрегування та факторизація», наданий файл `materials/Lab3.pdf`.
2. Катренко А. В., Пасічник В. В., Пасько В. П. Теорія прийняття рішень. Київ: Видавнича група BHV, 2009. Бібліографічний опис за методичкою.

Додаток. Лістинг програмної реалізації

```
#!/usr/bin/env python3
"""Агрегування та факторизація відношення за взаємною досяжністю."""
import argparse
import csv
import json
from html import escape
from pathlib import Path

from charts import generate

LAB_ROOT = Path(__file__).resolve().parent.parent


def composition(a, b):
    """Булева композиція: існує k з a[i][k] та b[k][j]."""
    n = len(a)
    return [[int(any(a[i][k] and b[k][j] for k in range(n)))
             for j in range(n)] for i in range(n)]


def properties(a):
    n = len(a)
    return {
        "reflexive": all(a[i][i] for i in range(n)),
        "symmetric": all(a[i][j] == a[j][i] for i in range(n) for j in
range(n)),
        "antisymmetric": all(i == j or not (a[i][j] and a[j][i]) for i
in range(n) for j in range(n)),
        "transitive": all(not (a[i][k] and a[k][j]) or a[i][j]
                          for i in range(n) for k in range(n) for j in
range(n)),
    }


def solve(base, student_number, shift_mode, max_powers):
    if not base or any(len(row) != len(base) for row in base):
        raise ValueError("Матриця повинна бути непорожньою та
квадратною")
    if any(value not in (0, 1) for row in base for value in row):
        raise ValueError("Елементи матриці мають дорівнювати 0 або 1")
    if student_number < 1:
        raise ValueError("Номер студента має бути додатним")
    n = len(base)
    shift = student_number - 1
    if shift_mode == "rows":
        k = shift % n
        p = [row[k:] + row[:k] for row in base]
    else:
        flat = [value for row in base for value in row]
        k = shift % (n * n)
        flat = flat[k:] + flat[:k]
        p = [flat[i * n:(i + 1) * n] for i in range(n)]
    current = p
```

```
    seen = {}
    powers = []
    closure = [[0] * n for _ in range(n)]
    for exponent in range(1, max_powers + 1):
        key = tuple(tuple(row) for row in current)
        powers.append({"exponent": exponent, "matrix": current})
        closure = [[int(closure[i][j] or current[i][j]) for j in
range(n)] for i in range(n)]
        if key in seen:
            first_occurrence = seen[key]
            repeated_at = exponent
            break
        seen[key] = exponent
        current = composition(current, p)
    else:
        raise ValueError("Межу степенів до першого повторення
перевищено; збільште --max-powers")
    reach = [[int(i == j or closure[i][j]) for j in range(n)] for i in
range(n)]
    mutual = [[int(reach[i][j] and reach[j][i]) for j in range(n)] for
i in range(n)]
    unassigned = set(range(n))
    classes = []
    while unassigned:
        first = min(unassigned)
        group = [i for i in sorted(unassigned) if mutual[first][i]]
        classes.append(group)
        unassigned.difference_update(group)
    quotient = [[int(any(p[i][j] for i in left for j in right))
                 for right in classes] for left in classes]
    condensation = [[int(i != j and quotient[i][j]) for j in
range(len(classes))]
                    for i in range(len(classes))]
    witnesses = [{"source_class": i + 1, "target_class": j + 1,
                  "original_edges": [[u + 1, v + 1] for u in
classes[i] for v in classes[j] if p[u][v]]}
                 for i in range(len(classes)) for j in
range(len(classes)) if quotient[i][j]]
    return {"student_number": student_number, "shift_count": shift,
"shift_mode": shift_mode,
            "base_matrix": base, "matrix": p, "powers": powers,
            "first_repeated_exponent": repeated_at,
"first_occurrence_exponent": first_occurrence,
            "period": repeated_at - first_occurrence,
"transitive_closure": closure,
            "reachability": reach, "mutual_reachability": mutual,
            "classes": [[i + 1 for i in group] for group in classes],
            "quotient": quotient, "condensation": condensation,
"factor_witnesses": witnesses,
            "properties": {"initial": properties(p),
"transitive_closure": properties(closure),
                           "reachability": properties(reach),
"mutual_reachability": properties(mutual),
                           "quotient": properties(quotient)}}
```

```
def write_matrix_csv(path, matrix):
    with path.open("w", encoding="utf-8", newline="") as stream:
        writer = csv.writer(stream)
        writer.writerow(["row/column", *range(1, len(matrix) + 1)])
        for i, row in enumerate(matrix, 1):
            writer.writerow([i, *row])


def write_dashboard(result, path):
    parts = ['<!doctype html><html lang="uk"><meta charset="utf-8">',
             '<meta name="viewport" content="width=device-width,
initial-scale=1">',
             '<title>Лабораторна 3 — агрегування та факторизація</
title>',
             '<style>body{font:18px/1.6 Literata,serif;max-
width:1000px;margin:32px auto;padding:0 24px;color:#17212b}
figure{margin:12pt 0}img{display:block;width:100%}figcaption{margin-
top:6pt;font-size:12pt;text-align:center}table{border-
collapse:collapse;margin:auto}th,td{border:1px solid
#d5dee8;padding:8px 18px;text-align:center}</style>',
             '<h1>Агрегування та факторизація</h1>',
             '<p>Біжик Юрій Володимирович · ФЕІ-34с · N=1</p>',
             '<p>' + escape(result['source']) + '</p>']
    for number, (key, title) in enumerate([
        ('matrix', 'Початкове відношення P'),
        ('transitive_closure', 'Транзитивне замикання'),
        ('reachability', 'Відношення досяжності'),
        ('mutual_reachability', 'Взаємна досяжність D'),
        ('quotient', 'Фактор-відношення P/D'),
    ], 1):
        matrix = result[key]
        rows = ''.join('<tr>' + ''.join(f'<td>{v}</td>' for v in row)
+ '</tr>' for row in matrix)
        parts.append(f'<figure><table>{rows}</
table><figcaption>Таблиця {number} — {title}</figcaption></figure>')
    for i, group in enumerate(result['classes'], 1):
        parts.append(f'<p>A{i} = {{{", ".join("x"+str(v) for v in
group)}}}</p>')
    for number, (filename, title) in enumerate([
        ('initial-graph.svg', 'Первісний граф'), ('relation-
matrices.svg', 'Матриці відношень'),
        ('factor-graph.svg', 'Фактор-граф'), ('condensation.svg',
'Конденсація без петель'),
    ], 1):
        parts.append(f'<figure><img src="figures/{filename}"
alt="{title}"><figcaption>Рисунок {number} — {title}</figcaption></
figure>')
    parts.append('<p>Фактор-граф може мати внутрішньокласову петлю.
Ациклічний граф конденсації отримується після її вилучення.</p></
html>')
    path.write_text('\n'.join(parts) + '\n', encoding='utf-8')


def main():
```

```
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--student-number", type=int, default=1)
    parser.add_argument("--input", type=Path, default=LAB_ROOT /
"data" / "input.json")
    parser.add_argument("--output-dir", type=Path, default=LAB_ROOT /
"data")
    parser.add_argument("--figures-dir", type=Path, default=LAB_ROOT /
"figures")
    parser.add_argument("--shift-mode", choices=["flat", "rows"],
default="flat",
                        help="Циклічний зсув всієї послідовності
елементів або кожного рядка")
    parser.add_argument("--max-powers", type=int, default=10000)
    args = parser.parse_args()
    try:
        data = json.loads(args.input.read_text(encoding="utf-8"))
        result = solve(data["base_matrix"], args.student_number,
args.shift_mode, args.max_powers)
        result["source"] = data.get("source", "Власна матриця з
вхідного JSON")
        args.output_dir.mkdir(parents=True, exist_ok=True)
        args.figures_dir.mkdir(parents=True, exist_ok=True)
        (args.output_dir /
"results.json").write_text(json.dumps(result, ensure_ascii=False,
indent=2) + "\n", encoding="utf-8")
        for key in ["matrix", "transitive_closure", "reachability",
"mutual_reachability", "quotient", "condensation"]:
            write_matrix_csv(args.output_dir / f"{key}.csv",
result[key])
        with (args.output_dir / "classes.csv").open("w",
encoding="utf-8", newline="") as stream:
            writer = csv.writer(stream)
            writer.writerow(["class", "vertices"])
            for i, group in enumerate(result["classes"], 1):
                writer.writerow([i, " ".join(f"x{value}" for value in
group)])
        generate(result, args.figures_dir)
        write_dashboard(result, LAB_ROOT / "index.html")
    except (ValueError, KeyError, TypeError, OSError) as exc:
        parser.exit(2, f"Помилка: {exc}\n")
    print(json.dumps(result, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    main()
```